



ARCOS

Sistemas Distribuidos
Grado en Ingeniería Informática

Práctica Final: Servicio de envío de mensajes distribuido

Miembros del grupo:

Alejandro Quirante Sanz - 100522183@alumnos.uc3m.es
Álvaro de la Roza Garcia - 100495871@alumnos.uc3m.es

2025/2026

Índice

1. Introducción	2
2. Diseño general	2
3. Estructura de datos del servidor	2
4. Protocolo de comunicación	3
4.1. Registro (REGISTER)	3
4.2. Baja (UNREGISTER)	3
4.3. Conexión (CONNECT)	3
4.4. Desconexión (DISCONNECT)	3
4.5. Envío de mensaje (SEND / SENDATTACH)	3
4.6. Entrega servidor → cliente (SEND_MESSAGE / SEND_MESSAGE_ATTACH)	3
4.7. ACK de entrega (SEND_MESS_ACK / SEND_MESS_ATTACH_ACK)	4
4.8. Usuarios conectados (USERS)	4
5. Implementación del servidor (C)	4
5.1. Mecanismo de entrega de mensajes pendientes	4
5.2. Comunicación con el servicio RPC	4
5.3. Generación de IDs de mensajes	4
6. Implementación del cliente (Python)	4
6.1. SocketUtils	4
6.2. ServerConnection	5
6.3. MessageReceiver	5
6.4. ChatClient	5
6.5. Normalización de mensajes (Servicio Web)	5
7. Implementación del servicio RPC (C)	5
8. Implementación del servicio web (Python)	6
9. Compilación y ejecución	6
9.1. Servidor C	6
9.2. Ejecución del servidor RPC	6
9.3. Ejecución del servidor de mensajería	6
9.4. Servicio web	6
9.5. Cliente Python	7
10. Pruebas realizadas	7
11. Conclusiones	11
12. Dificultades encontradas	11

1. Introducción

El objetivo de este proyecto ha sido diseñar e implementar un sistema de mensajería distribuido que permite a usuarios registrados intercambiar mensajes de texto de hasta 255 caracteres, así como archivos adjuntos de cualquier tamaño. La aplicación se compone de tres elementos principales: un servidor concurrente desarrollado en C que gestiona el registro, conexión y reenvío de mensajes entre usuarios; clientes desarrollados en Python que permiten a los usuarios interactuar con el sistema; y servicios auxiliares (RPC y Web) que amplían la funcionalidad del sistema.

La primera parte de la práctica implementa el núcleo del sistema de mensajería, incluyendo el registro de usuarios, conexión/desconexión, envío de mensajes de texto y consulta de usuarios conectados. El servidor es multihilo y capaz de almacenar mensajes pendientes para usuarios desconectados, entregándolos automáticamente cuando se conectan.

La segunda parte añade tres funcionalidades: transferencia de archivos adjuntos entre usuarios (P2P, sin intervención del servidor), un servicio web de normalización de mensajes (eliminación de espacios redundantes), y un servicio RPC que registra todas las operaciones realizadas por los usuarios en el servidor.

2. Diseño general

El sistema sigue una arquitectura cliente-servidor con los siguientes componentes:

Servidor de mensajería: Implementado en C con sockets TCP y multihilo. Gestiona el estado de los usuarios (registrados/conectados/desconectados), almacena mensajes pendientes y actúa como intermediario en el envío de mensajes de texto. Es el único componente que conoce la ubicación de todos los clientes.

Cliente de usuario: Implementado en Python con dos hilos: uno para la interfaz de usuario y envío de comandos, y otro para recibir mensajes entrantes de otros usuarios a través de un puerto de escucha. Se comunica con el servidor mediante sockets TCP y sigue el protocolo definido en el enunciado.

Servicio RPC: Servidor ONC-RPC que recibe notificaciones del servidor principal cada vez que un usuario realiza una operación (REGISTER, UNREGISTER, CONNECT, DISCONNECT, USERS, SEND, SENDATTACH). Imprime por pantalla el registro de la operación.

Servicio Web: API REST desarrollada con Flask que normaliza mensajes eliminando espacios redundantes. Es invocada por el cliente antes de enviar un mensaje.

Los clientes se comunican directamente entre sí para la transferencia de archivos (operación GETFILE), sin pasar por el servidor. Para ello, el cliente solicitante consulta al servidor la IP y puerto del usuario destino mediante el comando USERS, que ahora devuelve también la dirección de escucha de cada usuario conectado.

3. Estructura de datos del servidor

El servidor mantiene una estructura global de usuarios:

```
typedef struct {
    char    name[MAX_NAME];
    int     status;                // DISCONNECTED (0) o CONNECTED (1)
    char    ip[MAX_IP];           // IP del cliente cuando está conectado
    char    register_ip[MAX_IP];  // IP desde la que se registró
    char    port[MAX_PORT];       // Puerto de escucha del cliente
    unsigned int msg_counter;      // Contador para IDs de mensajes
}
```

3. Estructura de datos del servidor

```
    PendingMsg pending[MAX_PENDING]; // Mensajes pendientes
    int num_pending;
} User;
```

La lista de usuarios se protege con un mutex (`pthread_mutex_t users_mutex`) para garantizar la seguridad al acceder desde múltiples hilos, necesario para atender varias request al mismo tiempo. Los mensajes pendientes se almacenan en un array fijo dentro de cada usuario, con un límite de `MAX_PENDING` (100 mensajes pendientes por usuario). Cada mensaje pendiente contiene:

```
typedef struct {
    unsigned int id;
    char sender[MAX_NAME];
    char text[MAX_MSG];
    char filename[MAX_NAME]; // Para adjuntos (Parte 2)
    int has_attach;          // Booleano
} PendingMsg;
```

Esta estructura permite almacenar tanto mensajes de texto como mensajes con archivo adjunto, aunque el archivo en sí no se almacena en el servidor — solo el nombre del archivo, que se utiliza para notificar al destinatario qué archivo debe solicitar mediante `GETFILE`.

4. Protocolo de comunicación

Todos los mensajes entre cliente y servidor utilizan cadenas terminadas en `\0`. Cada operación abre una nueva conexión TCP, realiza el intercambio y la cierra. A continuación se detallan los mensajes implementados:

4.1. Registro (REGISTER)

1. Cliente → Servidor: "REGISTER" \0 <usuario>\0
2. Servidor → Cliente: <byte> (0=éxito, 1=usuario existe, 2=error)

4.2. Baja (UNREGISTER)

1. Cliente → Servidor: "UNREGISTER" \0 <usuario>\0
2. Servidor → Cliente: <byte> (0=éxito, 1=usuario no existe, 2=error)

4.3. Conexión (CONNECT)

1. Cliente → Servidor: "CONNECT" \0 <usuario>\0 <puerto>\0
2. Servidor → Cliente: <byte> (0=éxito, 1=usuario no existe, 2=ya conectado, 3=error)

4.4. Desconexión (DISCONNECT)

1. Cliente → Servidor: "DISCONNECT" \0 <usuario>\0
2. Servidor → Cliente: <byte> (0=éxito, 1=no existe, 2=no conectado, 3=error)

4.5. Envío de mensaje (SEND / SENDATTACH)

1. Cliente → Servidor: "SEND" \0 <remitente>\0 <destino>\0 <mensaje>\0 o "SENDATTACH" \0 <remitente>\0 <destino>\0 <mensaje>\0 <archivo>\0
2. Servidor → Cliente: <byte> (0=éxito) + si éxito: <id_mensaje>\0

4.6. Entrega servidor → cliente (SEND_MESSAGE / SEND_MESSAGE_ATTACH)

1. Servidor → Cliente: "SEND_MESSAGE" \0 <remitente>\0 <id>\0 <mensaje>\0 o "SEND_MESSAGE_ATTACH" \0 <remitente>\0 <id>\0 <mensaje>\0 <archivo>\0

4. Protocolo de comunicación

4.7. ACK de entrega (SEND_MESS_ACK / SEND_MESS_ATTACH_ACK)

1. Servidor → Cliente: "SEND_MESS_ACK" \0 <id>\0 o "SEND_MESS_ATTACH_ACK" \0 <id>\0 <archivo>\0

4.8. Usuarios conectados (USERS)

1. Cliente → Servidor: "USERS" \0 <usuario>\0
2. Servidor → Cliente: <byte> (0=éxito, 1=usuario no conectado, 2=error)
3. Si éxito: <num_usuarios>\0 + tantas cadenas como "usuario :: IP :: puerto\0"

5. Implementación del servidor (C)

El servidor se implementa en un único archivo `server.c`. Al iniciarse, crea un socket TCP en el puerto especificado, configura `SO_REUSEADDR` y entra en un bucle de aceptación de conexiones. Por cada conexión entrante, se crea un nuevo hilo que ejecuta `handle_client()`. El hilo es `detached` para que el servidor no tenga que esperar a su finalización. De esta forma usamos un modelo de hilos por solicitud, que presenta cierta ventaja a la hora de manejar muchos clientes, ya que con un hilo por cliente podría haber problemas intentando mantener tantos hilos, por lo que este modelo es de los más usados, mejorando la escalabilidad del sistema.

De forma similar podríamos haber implementado un `thread pool` que maneje las solicitudes en vez de levantar un hilo cada vez que llega una request, sin embargo, la mejora no es mucha y la implementación se vuelve más complicada.

5.1. Mecanismo de entrega de mensajes pendientes

Cuando un usuario se conecta exitosamente, el servidor recorre su lista de mensajes pendientes e intenta entregarlos uno a uno mediante la función `deliver_message()`. Si la entrega falla (el cliente destino no responde), el servidor lo marca como desconectado y detiene la entrega de los mensajes restantes. Solo se eliminan de la cola los mensajes entregados con éxito.

5.2. Comunicación con el servicio RPC

El servidor obtiene la IP del servicio RPC de la variable de entorno `LOG_RPC_IP`. Para cada operación, llama a `call_rpc_log()` que, mediante `clnt_create()`, establece una conexión con el servidor RPC y envía la operación correspondiente. Para las operaciones `SENDATTACH`, también envía el nombre del archivo. Los errores de comunicación RPC se ignoran silenciosamente para no interrumpir el servicio principal.

5.3. Generación de IDs de mensajes

Cada usuario mantiene un contador `msg_counter` de tipo `unsigned int`. Al enviar un mensaje, se incrementa el contador y se asigna ese valor como ID. Si el contador llega a 0 por desbordamiento, se reinicia en 1 (ya que el ID 0 no se utiliza). Esto garantiza un ciclo completo de IDs sin necesidad de persistencia.

6. Implementación del cliente (Python)

Hemos realizado un diseño modular para que la aplicación sea más limpia y fácil de mantener:

6.1. SocketUtils

Clase estática que proporciona métodos para enviar/recibir cadenas terminadas en `\0` y bytes individuales, así como para encontrar un puerto libre mediante `socket.bind('', 0)`.

6. Implementación del cliente (Python)

6.2. ServerConnection

Gestiona la comunicación con el servidor. Cada método (register, unregister, connect, disconnect, send_message, send_attach, get_users) abre una conexión al servidor, envía los datos según el protocolo y devuelve el resultado.

6.3. MessageReceiver

Hilo que escucha en el puerto asignado durante la conexión. Espera conexiones entrantes del servidor (para recibir mensajes) o de otros clientes (para la transferencia de archivos). Soporta los comandos:

- SEND_MESSAGE: mensaje de texto simple
- SEND_MESSAGE_ATTACH: mensaje con archivo adjunto
- SEND_MESS_ACK: confirmación de entrega
- SEND_MESS_ATTACH_ACK: confirmación con nombre de archivo
- GET_FILE: solicitud de transferencia de archivo (de otros clientes)

6.4. ChatClient

Clase principal que orquesta todas las operaciones. Mantiene un caché de usuarios conectados (users_cache) con su IP y puerto, actualizado mediante el comando USERS. Implementa los comandos:

- REGISTER <user>: registra un usuario
- UNREGISTER <user>: da de baja
- CONNECT <user>: busca puerto libre, inicia MessageReceiver, envía CONNECT al servidor
- DISCONNECT <user>: detiene MessageReceiver, envía DISCONNECT
- USERS: consulta y actualiza caché
- SEND <user> <msg>: envía mensaje (opcionalmente normalizado por web service)
- SENDATTACH <user> <msg> <file>: envía mensaje con archivo adjunto
- GETFILE <user> <remote> <local>: solicita archivo directamente a otro cliente
- QUIT: desconecta si es necesario y sale

6.5. Normalización de mensajes (Servicio Web)

El cliente contacta con el servicio web en `http://localhost:5000/normalize` enviando un JSON con el texto. Si el servicio responde correctamente, utiliza el texto normalizado; en caso contrario, envía el mensaje original. Esto asegura robustez ante caídas del servicio web.

7. Implementación del servicio RPC (C)

El servicio RPC se define en `log.x` con la siguiente interfaz:

```
struct log_operation_args {
    string user<>;
    string operation<>;
};

struct log_sendattach_args {
    string user<>;
    string operation<>;
    string filename<>;
};

program LOGPROG {
    version LOGVERS {
        int log_operation(log_operation_args) = 1;
```

7. Implementación del servicio RPC (C)

```
    int log_sendattach(log_sendattach_args) = 2;
} = 1;
} = 0x20000001;
```

El servidor RPC (`rpc_server.c`) implementa las funciones `log_operation_1_svc()` y `log_sendattach_1_svc()`, que simplemente imprimen por pantalla la información recibida y devuelven TRUE. El programa se registra en el portmapper y entra en `svc_run()`.

El archivo `CMakeLists.txt` y `services/CMakeLists.txt` automatizan la generación de los stubs RPC mediante `rpcgen` y la compilación del servidor y los clientes, garantizando la inclusión correcta de las cabeceras necesarias para estos stubs.

8. Implementación del servicio web (Python)

El servicio web se implementa con Flask en `web_service.py`:

```
@app.route('/normalize', methods=['GET'])
def normalize():
    data = request.get_json()
    text = data.get("text")
    normalized = " ".join(text.split())
    return jsonify({"result": normalized}), 200
```

Recibe un JSON con el campo `text`, elimina espacios redundantes usando `split()` y `join()`, y devuelve el texto normalizado.

9. Compilación y ejecución

A continuación se detallan los pasos para compilar y utilizar el código del servidor y de los clientes:

9.1. Servidor C

El servidor se compila mediante dos `CMakeLists`, para poder tener los archivos más ordenados y hacer el proyecto más fácilmente escalable, ya que en el `cmake` superior bastaría con incluir otro subdirectorio con las funcionalidades que se quieran añadir. Y además hacemos que los archivos estén más ordenados y los ejecutables se generen de forma más limpia.

Para compilar el servidor principal y el servidor RPC:

```
make clean #para limpiar posibles restos de anteriores compilaciones
cmake .
make
```

Esto genera:

- `services/build/server` (servidor de mensajería)
- `services/build/rpc_server` (servidor RPC)

9.2. Ejecución del servidor RPC

```
export LOG_RPC_IP=127.0.0.1
services/build/rpc_server
```

9.3. Ejecución del servidor de mensajería

```
services/build/server -p <puerto>
```

9.4. Servicio web

```
python3 web_service.py
```

9.5. Cliente Python

```
python3 client.py -s <IP_servidor> -p <puerto>
```

10. Pruebas realizadas

A continuación se presentan las pruebas realizadas sobre el sistema, cubriendo desde la funcionalidad básica de la Parte 1 hasta las ampliaciones de la Parte 2 (transferencia de ficheros, servicio web y RPC).

ID	Descripción / Acción	Resultado Esperado	Estado
T01	Ciclo completo: registro > conexión > envío > desconexión > baja	• Registro alice → REGISTER OK • Conexión alice → CONNECT OK • Envío «Hola bob!» → SEND OK - MESSAGE 1 • Desconexión alice → DISCONNECT OK • Baja alice → UNREGISTER OK El servidor muestra cada operación exitosa. Bob recibe el mensaje y Alice recibe el ACK correspondiente.	✓
T02	Mensajes encolados: envío a usuario offline > entrega al reconectar	• Alice (conectada) envía dos mensajes a Bob (offline) • Servidor almacena: MESSAGE 1 FROM alice TO bob STORED • Bob se conecta → recibe ambos mensajes automáticamente • Alice recibe SEND MESSAGE 1 OK y SEND MESSAGE 2 OK El sistema encola correctamente los mensajes y los entrega en orden al reconectar al destinatario.	✓
T03	USERS devuelve lista con formato «user :: IP :: puerto» y caché interna	• Alice y Bob conectados • Alice ejecuta USERS • Muestra: CONNECTED USERS (2 users connected) OK alice bob • La caché interna del cliente se puebla con formato {alice:(ip,port)} El cliente almacena correctamente la IP y puerto de los usuarios conectados para futuras transferencias P2P.	✓
T04	Incremento correcto de IDs: 3 mensajes secuenciales al mismo destinatario	• Alice envía «Primero», «Segundo», «Tercero» a Bob • IDs asignados: 1, 2, 3 • Bob recibe los mensajes con los IDs correctos • Si Carol envía a Bob, su ID empieza en 1 (independiente por remitente)	✓

10. Pruebas realizadas

ID	Descripción / Acción	Resultado Esperado	Estado
		El contador de mensajes es independiente por cada usuario remitente.	
T05	Baja con mensajes pendientes: UNREGISTER elimina la cola	• Bob offline, Alice le envía un mensaje • Servidor almacena mensaje pendiente • Se ejecuta UNREGISTER bob → OK • Bob se registra y conecta de nuevo → no recibe el mensaje anterior	✓
		Al dar de baja a un usuario, todos sus mensajes pendientes se eliminan del servidor.	
T06	Tres usuarios: alice > bob > carol, reenvíos cruzados	• Alice envía a Bob y a Carol • Bob envía a Carol • Carol recibe mensajes de Alice (ID 2) y de Bob (ID 1) • Todos los ACKs se entregan correctamente	✓
		El sistema maneja correctamente múltiples usuarios y conversaciones simultáneas.	
T07	Doble error: CONNECT de no-registrado + doble CONNECT del mismo usuario	• CONNECT fantasma → CONNECT FAIL, USER DOES NOT EXIST • CONNECT alice (primera vez) → CONNECT OK • CONNECT alice (segunda vez) → USER ALREADY CONNECTED • DISCONNECT alice → DISCONNECT OK • DISCONNECT alice (de nuevo) → DISCONNECT FAIL, USER NOT CONNECTED	✓
		El servidor valida correctamente el estado del usuario y rechaza operaciones inválidas.	
T08	SEND mensaje de exactamente 255 caracteres y SEND a sí mismo	• Enviar mensaje de 255 "A"s → SEND OK - MESSAGE 1 • Bob recibe exactamente 255 caracteres • SEND alice Hola yo misma → Alice recibe su propio mensaje	✓
		Se respeta el límite de 255 caracteres y se permite el envío a uno mismo (el mensaje se entrega a través del servidor).	
T09	SEND a usuario inexistente y con servidor caído (robustez del cliente)	• SEND noexist Hola → SEND FAIL, USER DOES NOT EXIST	✓

10. Pruebas realizadas

ID	Descripción / Acción	Resultado Esperado	Estado
		• Se para el servidor • SEND bob Hola → SEND FAIL (cliente no crashea) • REGISTER nuevo → REGISTER FAIL • Se reinicia el servidor → las operaciones vuelven a funcionar El cliente es robusto ante caídas del servidor y no crashea, mostrando el error apropiado.	
T10	SENDATTACH completo: envío, recepción y ACK con nombre de fichero	• Alice: SENDATTACH bob Mira este fichero /tmp/datos.txt • SENDATTACH OK - MESSAGE 1 • Bob recibe: MESSAGE 1 FROM alice Mira este fichero END FILE /tmp/datos.txt • Bob: GETFILE alice /tmp/datos.txt /tmp/recibido.txt → OK • Alice recibe ACK con nombre de fichero La transferencia de ficheros adjuntos funciona correctamente, tanto el envío como la recepción posterior.	✓
T11	SENDATTACH encolado: destinatario offline > recibe adjunto al conectar	• Bob offline, Alice envía SENDATTACH con fichero • Servidor almacena mensaje con has_attach=1 • Bob se conecta → recibe mensaje con END FILE /tmp/datos.txt • Bob puede hacer GETFILE para obtener el archivo Los mensajes con adjuntos se encolan correctamente y se entregan al reconectar el destinatario.	✓
T12	GETFILE fichero binario: imagen o ejecutable transferido byte a byte	• Transferir un archivo binario (imagen, ejecutable) • GETFILE completa la transferencia • Se verifica la integridad byte a byte (ej. con cmp o md5sum) La transferencia P2P es binaria segura y preserva el contenido exacto del archivo.	✓
T13	GETFILE con caché vacía y usuario desconectado entre medio	• Bob tiene la caché vacía • GETFILE alice ... → internamente llama a USERS para refrescar	✓

10. Pruebas realizadas

ID	Descripción / Acción	Resultado Esperado	Estado
		• Si se encuentra a Alice, la transferencia es OK • Alice se desconecta, Bob intenta GETFILE de nuevo • USERS silencioso ya no encuentra a Alice → FILE TRANSFER FAILED, user not connected El cliente refresca automáticamente la caché y maneja correctamente la desconexión del usuario destino.	
T14	Servicio web: normalización de espacios en SEND y SENDATTACH	• Servicio web corriendo en localhost:5000 • Alice envía: SEND bob "hola mundo que tal" • El cliente normaliza a "hola mundo que tal" (espacios simples) • Bob recibe el mensaje normalizado El servicio web elimina correctamente los espacios redundantes. Se verifica también con curl.	✓
T15	Web service caído: cliente usa el mensaje original (fallback robusto)	• Servicio web NO está corriendo • Alice ejecuta SEND bob hola mundo • Timeout de 2s, se captura la excepción • Cliente usa el mensaje original (sin normalizar) • El mensaje se envía correctamente y el cliente NO crashea El cliente es robusto ante la caída del servicio web y utiliza el mensaje original como fallback.	✓
T16	Servidor RPC: verifica que registra todas las operaciones	• Servidor RPC corriendo con LOG_RPC_IP=localhost • Cliente ejecuta: REGISTER alice, CONNECT alice, SEND bob Hola • Servidor RPC muestra: alice REGISTER, alice CONNECT, alice SEND • Para SENDATTACH también se muestra el nombre del fichero El servidor RPC registra correctamente todas las operaciones de los usuarios en el servidor de mensajería principal.	✓

10. Pruebas realizadas

Todas las pruebas se han ejecutado en el entorno de laboratorio con contenedores Docker en diferentes máquinas, verificando la correcta comunicación entre procesos distribuidos. El sistema ha superado todas las pruebas funcionales, de casos límite y de robustez.

11. Conclusiones

Se ha completado el desarrollo completo del sistema de mensajería distribuido, incluyendo la funcionalidad obligatoria de la Parte 1 (registro, conexión, mensajes de texto) y las ampliaciones opcionales de la Parte 2 (transferencia de archivos P2P, servicio web de normalización, servicio RPC de registro).

El servidor en C es robusto, maneja correctamente la concurrencia mediante hilos y mutex, y cumple con todas las especificaciones del protocolo. La integración con RPC mediante `rpcgen` ha sido sencilla gracias a los ejemplos proporcionados en los ejercicios evaluables.

El cliente en Python es modular y extensible. La separación en clases (`ServerConnection`, `MessageReceiver`, `SocketUtils`) facilita el mantenimiento. La implementación de la recepción de mensajes en un hilo separado y la caché de usuarios conectados permite una experiencia de usuario fluida.

La transferencia de archivos P2P evita sobrecargar al servidor y demuestra el uso de sockets para comunicación directa entre clientes. El servicio web y el servicio RPC añaden valor al sistema, demostrando la interoperabilidad entre diferentes tecnologías (sockets, RPC, REST).

Tanto la parte cliente como la de servidor incluyen un extensivo tratamiento de errores que facilita la tarea de encontrar el error tanto para los técnicos como para usuarios.

Todos los componentes se han probado en el entorno de laboratorio con múltiples contenedores Docker con direcciones IP distintas, verificando el correcto funcionamiento del sistema distribuido.

Esta práctica resulta muy completa y muy interesante para refrescar y poner en práctica todos los contenidos del curso, especialmente la segunda parte que, al implementar todo lo que hemos estado viendo de forma escueta, se hace más dinámica y liviana.

12. Dificultades encontradas

1. **Gestión de mensajes pendientes:** Inicialmente se eliminaban todos los mensajes pendientes al detectar un fallo en la entrega. Se corrigió para eliminar solo los entregados exitosamente.
2. **Desbordamiento del contador de IDs:** Se implementó la lógica de reinicio a 1 cuando el `unsigned int` llega a 0 por desbordamiento, garantizando IDs únicos cíclicos.
3. **Detección de desconexión del cliente:** En la entrega de mensajes pendientes, si `deliver_message()` falla, se marca al usuario como desconectado. Esto es necesario porque el cliente podría haberse caído sin enviar `DISCONNECT`.
4. **IP de registro vs IP de conexión:** Para cumplir con la Sección 8.4 de la Parte 1 (solo se puede desconectar desde la IP de registro), se almacena `register_ip` y se verifica en `DISCONNECT`. Previamente guardábamos la de conexión, causando un pequeño error de comprensión.
5. **Normalización con servicio web:** La dependencia del servicio web podría hacer fallar el envío de mensajes si el servicio no responde. Se implementó un fallback que envía el mensaje original.
6. **Problemas con cmake:** Tuvimos algunos problemas para ajustar el directorio de salida al tener el `cmake` estructurado en dos niveles, así como para generar correctamente los archivos del `rpc` y que se borrasen los anteriores.

12. Dificultades encontradas

Todos los materiales necesarios para reproducir la compilación y ejecución se entregan en el fichero comprimido, incluyendo código fuente, CMakeLists, Makefile y documentación.